Stateful Workloads on Kubernetes

stable names, sticky disks, and software that knows how to run a database

Watch the 4-min video

> **In one line:** a StatefulSet gives each pod a stable name, ordered startup and its own persistent volume; an **operator** adds the database know-how (replication, failover, backups, upgrades). Prefer a managed database unless you need portability, a missing capability, or you have the expertise.

1 Stateless is easy, state is not

Kubernetes treats pods as replaceable: kill one, start another anywhere, nothing is lost. A database breaks every part of that. Each replica holds **its own data**, replicas have **roles** (one primary, several followers), peers must find **specific** members by name, and losing a disk can mean losing data. Running state on Kubernetes needs different building blocks, and an honest decision about whether to do it at all.

2 StatefulSet: identity that survives restarts

	Deployment	StatefulSet
Pod names	random suffix (`api-7d9f-x2k`)	ordinal: `kafka-0`, `kafka-1`, `kafka-2`
Storage	usually none, or one shared volume	`volumeClaimTemplates`: one PVC per pod, re-attached to the same ordinal
Network identity	one Service IP for all pods	headless Service gives each pod its own DNS name
Start / update order	any order, in parallel	0→N on create; updates N→0, one at a time, waiting for Ready
On scale-down / delete	pods and anything they wrote are gone	PVCs are **kept** by default, so data survives a mistake

A **headless Service** (`clusterIP: None`) doesn't load-balance; DNS returns the pods themselves, and each pod gets a record like `kafka-1.kafka-hl.shop.svc.cluster.local`. Clients can bootstrap from any broker, and peers can say "replicate from kafka-0". `podManagementPolicy: Parallel` drops the ordering for systems that don't need it, and an update `partition` lets you upgrade only the highest ordinals first, as a canary.

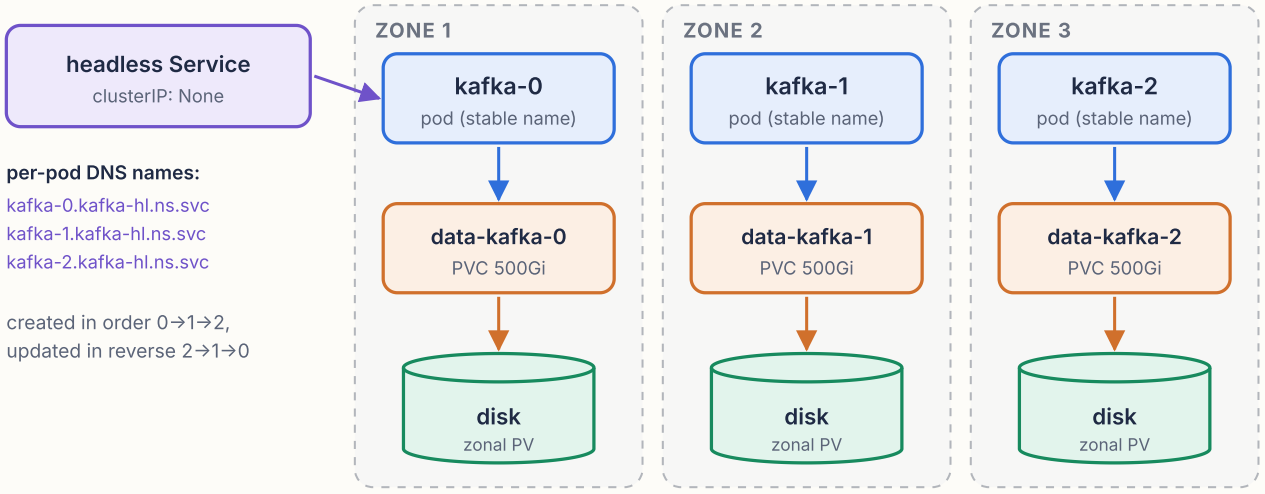


Figure 1. A three-broker StatefulSet spread over zones. Each pod keeps its name and its PVC; if `kafka-1` is rescheduled, it comes back as `kafka-1` with the same disk.

3 Persistent volumes and storage classes

A pod asks for storage with a **PersistentVolumeClaim** ("500 Gi, ReadWriteOnce, class premium-ssd"). A **StorageClass** names a CSI driver (Azure Disk, AWS EBS, GCE PD, Ceph…) and parameters; it **dynamically provisions** a cloud disk and binds it as a **PersistentVolume**. Settings that matter:

- `volumeBindingMode: WaitForFirstConsumer`: create the disk only after the pod is scheduled, in that pod's zone. Without it a disk can be created in zone 1 for a pod placed in zone 2.
- **Zonal disks:** most block disks live in one zone. If the pod moves, the disk is detached and re-attached on the new node, but only within that zone. A full zone outage leaves the pod stuck until the zone returns, so the database's own replication across zones is what protects you.
- `reclaimPolicy`: `Delete` (the default for dynamic volumes) removes the disk when the PV is released; `Retain` keeps it. Use Retain for anything precious.
- `allowVolumeExpansion: true` lets you grow a PVC in place. Access modes: RWO (one node), RWX (many nodes, file storage), RWOP (exactly one pod).

4 Operators: the know-how in software

Starting pods is the easy 10%. Someone must configure replication, **elect a new primary** on failure, take and test **backups**, perform **version upgrades** safely, rotate certificates and rebalance data. An **operator** encodes that runbook in a controller: you declare a **custom resource** (`kind: Kafka`, `kind: Cluster` for Postgres), and the controller runs the same reconcile loop as built-in controllers: observe, compare with the spec, act, repeat.

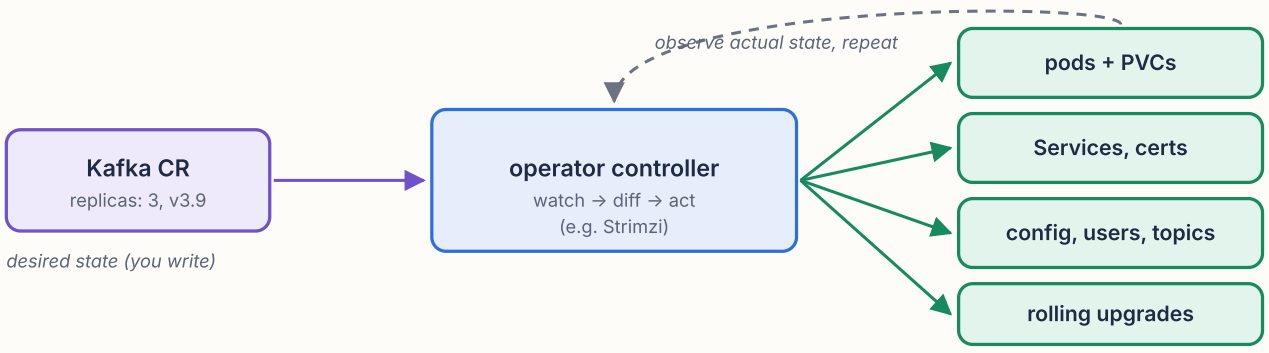


Figure 2. The operator pattern: a custom resource is the desired state; the controller creates and keeps fixing everything underneath it.

Mature examples: **Strimzi** (Kafka), **CloudNativePG**, Crunchy PGO and Zalando's operator (PostgreSQL), **Vitess** (sharded MySQL), and vendor operators for MongoDB, Redis and Elasticsearch. Quality varies widely; the Operator Framework's capability levels run from 1 (basic install) to 5 ("auto pilot": auto-scaling, auto-tuning). Check that failover, backup/restore and upgrades are actually automated before you trust one.

5 Should you run databases on Kubernetes?

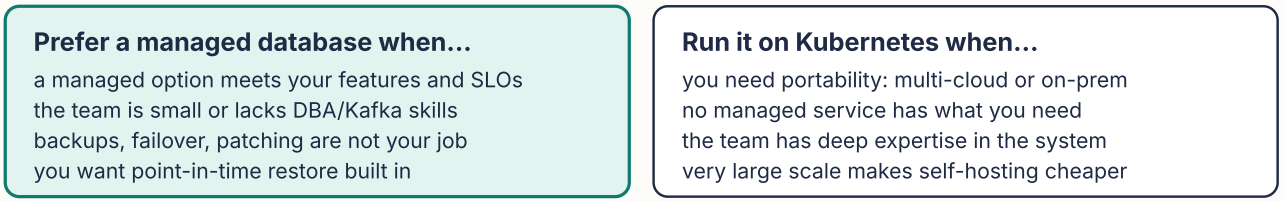


Figure 3. The decision is mostly about people and capabilities, not technology.

A managed database (RDS, Azure Database for PostgreSQL, Cloud SQL, Event Hubs, MSK...) costs more per CPU, but the provider runs backups with point-in-time restore, failover, patching and storage growth, and it does them every day for thousands of customers. Your team's time goes to the product. The costs of self-running are mostly invisible until a bad night: a failed upgrade, a full disk, a restore nobody has practised. Compare the total cost of ownership, including people (lesson 65).

6 If you do run state on Kubernetes

- **Use a mature operator** rather than hand-written StatefulSets.
- **Spread replicas across zones** with topology spread constraints or anti-affinity, and make the system itself replica-aware (Kafka `rack`, Postgres synchronous standby in another zone).
- **PodDisruptionBudgets** (e.g. `maxUnavailable: 1`) so node drains and upgrades (lesson 63) never take down two replicas at once.
- **Fast storage classes** with Retain, monitoring of disk usage and IOPS, and alerts well before full.
- **Backups outside the cluster** (object storage), and **regular restore tests**. A backup you've never restored is a hope, not a backup.
- Generous `terminationGracePeriodSeconds` for clean shutdown, a written **recovery runbook**, and dedicated node pools if noisy neighbours hurt latency.

WORKED EXAMPLE: KAFKA WITH THE STRIMZI OPERATOR

The team installs Strimzi and applies a `Kafka` resource (below): three brokers, 500 Gi each, rack-aware across zones, replication factor 3 with `min.insync.replicas: 2`. Strimzi creates the broker pods with stable names, one PVC each, the bootstrap and per-broker Services, a cluster CA and TLS certificates, and the topic and user operators. (Recent Strimzi versions manage pods through their own `StrimziPodSet` resource instead of StatefulSets, with the same guarantees: stable identity, one volume per broker.) To upgrade, they change `version`. The operator rolls brokers **one at a time**, and before restarting each one it checks that no partition would drop below its min in-sync replicas, so producers with `acks=all` keep writing throughout.

```
apiVersion: kafka.strimzi.io/v1beta2
kind: Kafka
metadata: { name: events }
spec:
  kafka:
    version: 3.9.0           # change this to upgrade
    replicas: 3
    rack: { topologyKey: topology.kubernetes.io/zone }
    storage: { type: persistent-claim, size: 500Gi, class: premium-ssd }
    config: { default.replication.factor: 3, min.insync.replicas: 2 }
```

Figure 4. The whole cluster as one custom resource. The operator turns it into dozens of Kubernetes objects.

COMMON MISTAKES

- Running a database as a Deployment: pods share or lose volumes and get random names.
- Immediate volume binding, so disks and pods end up in different zones.
- No PodDisruptionBudget: a node pool upgrade drains two of three replicas together.
- Backups stored in the same cluster or never restored in a test.

7 Key terms

StatefulSet: workload with stable pod names, ordered rollout, per-pod PVCs

PVC / PV: a claim for storage / the actual provisioned volume

Operator: controller + custom resource that automates running a system

Headless Service: DNS returns individual pod addresses, no virtual IP

StorageClass: recipe for dynamically provisioning volumes via a CSI driver

PodDisruptionBudget: limit on voluntary evictions at one time

8 Interview questions

Q1. Should you run databases on Kubernetes? BASIC

Prefer managed databases when they meet your needs, since they handle backups, failover and patching; run state on Kubernetes for portability, a missing capability or strong expertise, using StatefulSets, persistent volumes and a mature operator.

Q2. What does an operator do? BASIC

It automates operational knowledge such as failover, upgrades and backups through a custom controller that reconciles a custom resource.

Q3. What does a StatefulSet give you that a Deployment doesn't? BASIC

Stable ordinal pod names and DNS records, a persistent volume per pod that follows it across restarts, and ordered, one-at-a-time rollout.

Q4. A zone goes down. What happens to kafka-1, whose disk was in that zone? SCENARIO

The pod can't start elsewhere because its zonal disk can't attach in another zone; it stays Pending, and the cluster keeps serving from the other two brokers' replicas until the zone returns.

Q5. You're upgrading the node pool under a 3-node Postgres cluster. How do you avoid an outage? SCENARIO

A PDB with maxUnavailable 1 so drains go one node at a time, an operator that switches over the primary before its node is drained, and replicas spread across zones.

Q6. You scaled a StatefulSet from 3 to 2 and back to 3. Is kafka-2's data gone? TRICKY

No: by default its PVC is retained on scale-down, so the new kafka-2 re-attaches the same volume (unless the retention policy is set to delete).

Q7. Why can a pod and its new disk end up in different zones? TRICKY

With Immediate binding the disk is provisioned before scheduling, possibly in another zone; WaitForFirstConsumer provisions it where the pod is placed.

Next: SD-V-010 · Kubernetes Networking backendroadmap.com